

Cache Assignment in gem5

Homework 3

Part 2: Profiling

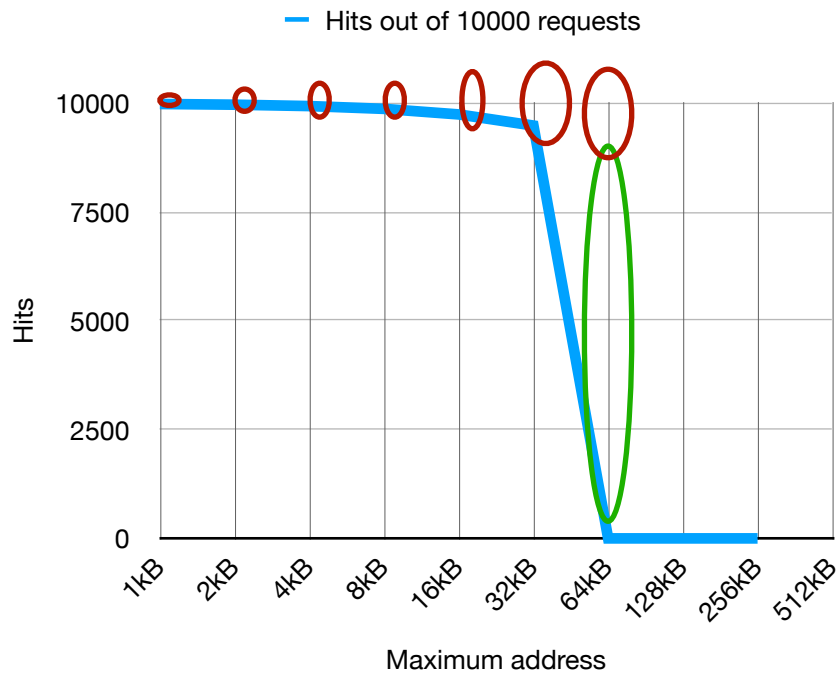
A: MYSTERY CACHE

CACHE SIZE:

My strategy is also to go through the blocks one by one. I'll change the maximum address:

- if the cache is bigger than the address space, there will only be compulsory misses (the first time we access an address).
- if the cache is smaller than the address space, there will be evictions, that will cause a jump on misses.

The cache size is 32kB.



Circled in red: compulsory misses

Circled in green: evictions causing misses

ASSOCIATIVITY:

I'll first read from the first address, and stride to always come back to the first set (a stride equal to the cache size).

If associativity is 1, one strides of cache size will create the first eviction+miss. We check that the eviction occurred by striding back to the set and stepping out of the max_address that makes look into the address 0.

If associativity is 2, two strides of cache size will create the first eviction+miss.

If associativity is i , i strides of cache_size will create the first eviction+miss.

To test associativity i ,

```
{
  "mode": "strided",
  "max_addr": "cache_size *(i+1)",
  "num_reqs": 1+1+i,
  "req_size": 64,
  "stride": cache_size //always stay in the same set
}
```

We increase i until we find that we had to evict a block, which is the LRU, so at address 0. This is detected by only misses (otherwise, there is only one hit, which is address 0).

The associativity of the cache is 8.

B: REAL CACHE PROFILING

In real life, caches are interconnected with other components, and dependant on the requests they receive,

- prog1 is incrementing variables at random addresses, but without changing the variable addresses. So it causes a very low instruction hit (we don't know what will be incremented) compared to a high data hit (we had the data loaded in cache and ready to use).

system.l2cache.overallHits::cpu.inst	7
system.l2cache.overallHits::cpu.data	27046

- prog2 is accessing variables that are next to each other in memory

system.l2cache.overallHits::cpu.inst	12
system.l2cache.overallHits::cpu.data	560

- prog2 is accessing variables that are far (blocks away) from each other in memory

system.l2cache.overallHits::cpu.inst	7
system.l2cache.overallHits::cpu.data	29

The broad picture of profiling a cache in a complex interconnected system is more complicated than giving a number for associativity and size, but these can be replaced by weaker attributes, such as being fast for high locality data (linked with a high associativity) or good with random data

(linked with a smaller associativity that allows for smaller miss penalties). The resulting tradeoff is between

- big associativity: small delays for high locality data but big delay for misses.
- small associativity: delays not as high in case of miss, but lower probability of hits because the sets are smaller.

A bigger cache size is allowing for less eviction but costs more hardware at constant delay, or can increase the delay at constant hardware.

Caches need to have their associativity optimised for their use: it depends on the type of programs running (as seen in prog1,2,3) but also their position in the memory hierarchy.

The instructions caches are very likely to have a high locality as programs are for the most part written linearly, while data can be sometimes accessed randomly or have a high locality, depending on the workload.

Reflexion

A :

The hardware can implement a sequential comparison (similar to the code) that would cause a long delay (increasing linearly with the size of the cache). Another approach, that necessitates more hardware surface (more expensive consumes more power) is a parallel comparison of tags/addresses, that allows for a search in constant time.

B :

Memory traces

SUBOPTIMAL	Block1	Block2	Block3	Block4
read 1	value1			
read 2	value1	value2		
read 3	value1	value2	value3	
read 4	value1	value2	value3	value4
read5 =evicts LRU =1	value5	value2	value3	value4
read 1	suboptimal because evicting 2, 3 or 4 would have kept 1			

OPTIMAL	Block1	Block2	Block3	Block4
read 1	value1			
read 2	value1	value2		
read 3	value1	value2	value3	
read 4	value1	value2	value3	value4
read5 =evicts LRU =1	value5	value2	value3	value4
read6=evicts LRU =2	value5	value6	value3	value4
read7=evicts LRU =3	value5	value6	value7	value4
read4	optimal because we evicted every other block but 4's			

C-1:

To model way prediction in gem5 we would need to reproduce in code the prediction model and account for the predictions misses that yield an additional delay, as a potential additional field.

C-2:

To model energy consumption of way prediction we would have to take into account the length of every wire that can depend on the relative position on the chip of each memory bit relative to the other. It is extremely difficult to predict.

The material used could also impact the energy dissipation (but it is reasonable to assume that the wires are made out of the same material).

It is probable that this depends on the program being executed, as the predicted data could be the one that has to use the shortest path or the longest, making it have different power consumption/ heat dissipation.

Among many other parameters to consider, the ideal comparison must include:

- physical placing of every component of the circuit (exact, microscopically)
- material used for each (almost at atomic scale)
- a predefined workload of data access, that has to be representative of the usage of the chip
- environment external influences: room temperature, computer temperature, motherboard temperature...

As a conclusion it is nearly impossible to validate the model to experiments at such a scale, so it is very hard to benchmark the model relative to the only truth which is experimentation

Still models can help design tradeoff for chip design, but probably at a broader scale, like considering the whole computer for example.

D-1:

Parallel actions (hit/miss) one solution could be to be blocking from before the first to after the last. As it is parallel, the delay would only be the hardware delay for one, but the slowest.

Another way of dealing with this would be to have the cache controller to keep track of signals from each parallel unit (pending...) and the miss or hit status in registers for each unit, acknowledging for dependencies.

D-2:

The maximum parallelism in a cache is increasing with the number of MSHR registers. For one unit occupied with a miss, every additional MSHR register is allowing for a parallel execution to keep going independently.